Name: G SHIVANI

Roll no: 2520090046

PRACTICAL WEEK-2

Session-2

Task1: Develop a C program that uses the system calls open(), read(), write(), and close() to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

```
  GNU nano 8.7.1                                  file_copy.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int source_fd, dest_fd;
    char buffer[1024];
    ssize_t bytes_read, bytes_written;

    source_fd = open("source.txt", O_RDONLY);

    if (source_fd == -1) {
        perror("Error opening source file");
        return 1;
    }

    dest_fd = open("destination.txt",
                   O_WRONLY | O_CREAT | O_TRUNC,
                   0644);

    if (dest_fd == -1) {
        perror("Error opening destination file");
        close(source_fd);
        return 1;
    }

    while ((bytes_read = read(source_fd, buffer, sizeof(buffer))) > 0) {

        bytes_written = write(dest_fd, buffer, bytes_read);

        if (bytes_written != bytes_read) {
            perror("Error writing to destination file");
            close(source_fd);
            close(dest_fd);
            return 1;
        }
    }

    if (bytes_read == -1) {
        perror("Error reading source file");
    }

    close(source_fd);
    close(dest_fd);

    printf("File copied successfully.\n");

    return 0;
}
```

```
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$ mkdir -p practical/week2/task1
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$ cd practical/week2/task1
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$ pwd
/home/shivani/OSSP/practical/week2/task1
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$ ls
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$ nano file_copy.c
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$ echo "This is a sample file
for demonstrating Linux system calls." > source.txt
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$ cat source.txt
This is a sample file for demonstrating Linux system calls.
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$ gcc file_copy.c -o file_cop
y
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$ ls
file_copy  file_copy.c  source.txt
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$ ./file_copy
File copied successfully.
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$ cat destination.txt
This is a sample file for demonstrating Linux system calls.
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$ ls -l
total 28
-rw-r--r--  1 shivani shivani    60 Aug  9 16:34 destination.txt
-rwxr-xr-x  1 shivani shivani 16216 Aug  9 16:34 file_copy
-rw-r--r--  1 shivani shivani  1082 Aug  9 16:33 file_copy.c
-rw-r--r--  1 shivani shivani    60 Aug  9 16:33 source.txt
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$
```

Observation: The program successfully copied the contents of source.txt to destination.txt using the open(), read(), write(), and close() system calls. The contents of both files were verified to be identical. During execution, the user-space program requested file operations from the Linux kernel through system calls, and the kernel performed the required file operations before returning control to the user-space program.

Task2: Use the strace utility to trace all system calls generated by the command cat sample.txt. Analyze the sequence of system calls and identify the kernel services involved.

```
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task1$ cd ~/OSSP
shivani@LAPTOP-ER8MCG0T:~/OSSP$ mkdir -p practical/week2/task2
shivani@LAPTOP-ER8MCG0T:~/OSSP$ cd practical/week2/task2
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task2$ pwd
/home/shivani/OSSP/practical/week2/task2
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task2$ echo "Hello, this is a sample file for system
call tracing." > sample.txt
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task2$ cat sample.txt
Hello, this is a sample file for system call tracing.
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task2$ strace --version
strace -- version 6.19
Copyright (c) 1991-2026 The strace developers <https://strace.io>.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
Optional features enabled: stack-trace=libunwind stack-demangle m32-mpers mx32-mpers
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task2$ strace  cat sample.txt
execve("/usr/bin/cat", ["cat", "sample.txt"], 0x7ffca1cb54d8 /* 26 vars */) = 0
brk(NULL)                               = 0x625e7099b000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7a269cc4c000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20763, ....}) = 0
mmap(NULL, 20763, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7a269cc46000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1",  O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\60\0\0\0\0\0\0\0", 832) = 832
...
+++ exited with 0 +++
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task2$
```

```
read(3, "02be481b2000-02be48244000 rw-p 0"..., 1024) = 1024
read(3, "c77c0000-7a369cce7f000 rw-p 00000000"..., 1024) =
read(3, "00 00:30 13631"..., 1024)          = 1024
read(3, "\n"..., 1024) = 1
read(3, "0 13625        /usr/lib/x86_64-linux-"..., 1024) = 566
close(3) = 0
sched_getaffinity(1365, 32, {0, 2, 8, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}) = 0
rt_sigaction(SIGSEGV, NULL, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rigaltstack(NULL, {ss_sp=NULL, ss_flags=SS_DISABLE, ss_size=0}, NULL, 16) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x7a269cc4000
mprotect(0x7a269cc40000, 4096, PROT_NONE) = 0
sigaltstack({ss_sp=0x7a269cc41000, ss_flags=0, ss_size=8192}, NULL) = 0
gettid() = 1365
rt_sigaction(SIGSEGV, {sa_handler=0x625e40d4790c, sa_mask=[], sa_flags=SA_RESTORER|
SA_ONSTACK|SA_SIGINFO, sa_restorer=0x7a269cc470c0}, NULL, 8) = 0
rt_sigaction(SIGBUS, NULL, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGSEGV, {sa_handler=0x625e40d4770c, sa_mask=[], sa_flags=SA_RESTORER|
SA_ONSTACK|SA_SIGINFO, sa_restorer=0x7a269cc470c0}, NULL, 8) = 0
prctl(PR_SET_MM, PR_SET_MM_ARG_START, 0x7ffd122ffdc0, 0, 0) = 0
statx(AT_FDCWD, "/usr/share/locale/cat", AT_STATX_SYNC_AS_STAT, STATX_ALL,
  {stx_mask=STATX_ALL|STATX_MNT_ID, stx_attributes=0, stx_mode=S_IFDIR|0755, stx_si
ze=4096, ... }) = 0
statx(AT_FDCWD, "/usr/share/locale/ucore", AT_STATX_SYNC_AS_STAT, STATX_ALL,
  {stx_mask=STATX_ALL|STATX_MNT_ID, stx_attributes=0, stx_mode=S_IFDIR|0755, stx_si
ze=4096, ...}) = 0
openat(AT_FDCWD, "/usr/share/coreutils/locale/ucore/en-US.fti", O_RDONLY|O_CLOEXEC) = 3
read(3, "", 1) = 0
statx(AT_FDCWD, "/usr/share/coreutils/locale/cat", AT_STATX_SYNC_AS_STAT, STATX_ALL,
  {stx_mask=STATX_ALL|STATX_MNT_ID, stx_attributes=0, stx_mode=S_IFDIR|0755, stx_si
ze=4096, ...}) = 0
openat(AT_FDCWD, "/usr/share/coreutils/locale/cat/en-US.fti", O_RDONLY|O_CLOEXEC) = 3
openat(ENOENT (No such file or directory), 0x7a269cc13000, 0) = -1 ENOENT (No such file or directory)
rt_sigaction(SIGPIPE, {sa_handler=SIG_DFL, sa_mask=[PIPE], sa_flags=SA_RESTORER|SA_
RESTART, sa_restorer=0x7a269cc470c0}, {sa_handler=SIG_IGN, sa_mask=[], sa_flags=
SA_RESTORER|SA_RESTART, sa_restorer=0x7a269cc470c0}, 8) = 0
rt_sigaction(SIGWINCH, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=SA_RESTORER|SA_RE
START, sa_restorer=0x7a269cc470c0}, {sa_handler=0x625e40d471c1, sa_mask=[], sa_flags=
SA_RESTORER|SA_ONSTACK|SA_SIGINFO, sa_restorer=0x7a269cc470c0}, 8) = 0
```

```
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task2$ strace -o trace.txt cat sample.txt
Hello, this is a sample file for system call tracing.
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task2$ ls
sample.txt  trace.txt
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task2$ cat trace.txt
execve("/usr/bin/cat", ["cat", "sample.txt"], 0x7ffc740e111d8  /* 26 vars */) = 0
brk(NULL)                               = 0x5c822091e000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS,  -1, 0) = 0x7647b72b9000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD,  "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20763,  ...}) = 0
mmap(NULL, 20763, PROT_READ, MAP_PRIVATE, 3, 0) =  0x7647b72b3000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\03\0>\0\1\0\0\0\0\0\0\1\0\0\0", 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=199120, ...}) = 0
mmap(NULL, 206288, PROT_READ, MAP_PRIVATE|MAP_DENYWRIITE, 3, 0) = 0x7647b7280000
mmap(0x7647b7287000, 135168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x70000) = 0x7647b7287000
mmap(0x7647b72a8000, 28672, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7647b72a8000
mmap(0x7647b72af000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2f000) = 0x7647b72af000
mmap(0x7647b72b1000, 5584,  PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7647b72b1000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3

...

read(3, "02be481b2000-02be48244000 rw-p 0"..., 1024) = 1024
read(3, "c77c0000-7a369cce7f000 rw-p 00000000"..., 1024) = 1024
read(3, "00 00:30 13631"..., 1024)          = 1024
read(3, "\n"...,  1024)                     = 1
read(3, "0 13625         /usr/lib/x86_64-linux-"...,  1024) =  566
+++ exited with 0 +++
shivani@LAPTOP-ER8MCG0T:~/OSSP/practical/week2/task2$
```

Observation: The strace utility successfully traced the system calls generated by the cat sample.txt command. The trace showed openat() being used to open sample.txt, followed by file and terminal operations using splice(), pipe(), and fcntl(). The close() system call was used to close the file descriptors, and exit_group(0) indicated successful program termination. The observed system calls demonstrate how the cat command interacts with Linux kernel services to access and display file contents.